Architecture logicielle et microservices

Informatique — Licence 3 (L3)

Cours INF-L3-16

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer du monolithe aux microservices en connaissance de cause : découpage en domaines, communication synchrone et par messages, données distribuées, résilience, observabilité et migration progressive — sans oublier les cas où le monolithe reste le meilleur choix.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Architectures monolithiques : forces et limites
• Leçon 2 — Découper un système en services : domaines et frontières
• Leçon 3 — Communication entre services : synchrone et par messages
• Leçon 4 — Gestion des données dans un système distribué
• Leçon 5 — Résilience : tolérance aux pannes et files d'attente
• Leçon 6 — Observabilité d'un système distribué
• Leçon 7 — Étude de cas : migration progressive d'un monolithe
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Il y a une quinzaine d'années, presque toutes les applications web naissaient sous la même forme : un seul programme, une seule base de données, un seul déploiement. Puis les plateformes de commerce, de vidéo et de transport ont grandi jusqu'à employer des centaines de développeurs, et l'architecture en services indépendants — les microservices — s'est imposée dans les grandes organisations. Depuis, les étudiants entendent tout et son contraire : le monolithe serait dépassé, les microservices seraient toujours supérieurs. La réalité est plus nuancée, et ce cours t'apprend précisément à la raisonner : un choix d'architecture est un arbitrage entre des avantages et des coûts, jamais une question de mode ou de prestige technique. Savoir défendre cet arbitrage devant une équipe, un client ou un jury est d'ailleurs une compétence d'ingénieur autant que de développeur.

Nous suivrons une étude fil rouge : SahelMarché, une plateforme fictive de commerce en ligne qui livre des produits dans tout le Mali. Elle a démarré, comme la plupart des projets réels, sous la forme d'un monolithe tout à fait honorable. À chaque leçon, nous examinerons une force ou une douleur de son architecture actuelle, puis la réponse que les microservices y apportent — avec les coûts propres du distribué : un réseau qui peut tomber en panne, des données éclatées entre services, une exploitation quotidienne plus lourde. Tu verras que découper un système est la partie la plus facile ; faire vivre le système découpé, chaque jour, est la vraie compétence d'architecte. Tu apprendras au passage le vocabulaire précis du domaine — contexte délimité, saga, disjoncteur, observabilité — car on ne discute bien que ce que l'on sait nommer

correctement.

Ce cours prolonge le développement d'applications web avancées (INF-L3-03), la conception logicielle et les patrons de conception (INF-L3-04) ainsi que le cours DevOps et intégration continue (INF-L3-12). Il suppose que tu sais ce qu'est une API REST, une transaction de base de données et un conteneur Docker. À la fin, tu seras capable de lire un schéma d'architecture distribuée avec un œil critique, de justifier un découpage en domaines, de concevoir une saga complète, et surtout de décider — arguments chiffrés à l'appui — quand les microservices sont la bonne réponse et quand un monolithe modulaire proprement construit suffit largement au besoin. Garde aussi l'esprit critique face aux récits spectaculaires des très grandes plateformes mondiales : leurs problèmes d'échelle ne sont pas forcément ceux des systèmes que tu construiras au Mali ou ailleurs.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire l'architecture d'un monolithe, ses forces réelles et les limites qui apparaissent quand le système et les équipes grandissent.
- Découper un système en services à partir des domaines métier et des contextes délimités, et justifier chaque frontière retenue.
- Choisir entre communication synchrone et communication par messages selon le besoin, en mesurant les conséquences de chaque mode.
- Expliquer la gestion des données distribuées : base de données par service, cohérence éventuelle et patron saga avec compensations.
- Mettre en place les mécanismes de résilience essentiels : délais d'attente, nouvelles tentatives, disjoncteur et files d'attente.
- Rendre un système distribué observable : journaux structurés, identifiant de corrélation, métriques et traces distribuées.
- Conduire la migration progressive d'un monolithe vers des services par le patron de l'étrangleur, étape par étape et sans arrêt de service.

Leçon 1 — Architectures monolithiques : forces et limites

Une application monolithique est une application construite et déployée comme une seule unité : toutes les fonctions métier — comptes clients, catalogue, commandes, paiement, livraison — vivent dans le même code source, s'appellent directement en mémoire et partagent une base de données unique. SahelMarché a démarré ainsi, et c'était le bon choix : trois développeurs, un seul artefact à produire, un seul serveur à louer. Le mot « monolithe » a pris une connotation péjorative injuste : il décrit une forme de déploiement, pas une qualité de conception. Un monolithe peut être proprement structuré en modules internes bien séparés, et nous verrons que cette variante, le monolithe modulaire, reste une option tout à fait légitime.

Les forces du monolithe sont concrètes. Développer est simple : un seul environnement à installer, un débogueur qui traverse tout le code, des refontes qui se

font par recherche et remplacement plutôt que par renégociation de contrats entre services. Tester est plus sûr : un test de bout en bout tourne en local, sans monter dix services. Les transactions de base de données offrent la cohérence forte sans effort : débiter un stock et enregistrer un paiement réussissent ou échouent ensemble, atomiquement. Enfin, l'exploitation est économique : un processus à superviser, un journal à lire, une sauvegarde à planifier. Pour une petite équipe et un produit encore jeune, ces avantages sont décisifs.

Les limites apparaissent avec la croissance, et elles sont organisationnelles autant que techniques. Quand SahelMarché atteint vingt développeurs, chacun déploie dans les pieds des autres : une livraison du module Paiement impose de redéployer tout le système, y compris le Catalogue qui n'a pas changé. La montée en charge est tout ou rien : si seul le catalogue souffre pendant les soldes, il faut dupliquer aussi les modules qui dorment. Un bogue mémoire dans un module peut faire tomber l'ensemble. Surtout, la base de code devient intimidante pour les nouveaux arrivants. Retiens la formulation honnête : le monolithe ne « casse » pas à une taille précise ; ses coûts de coordination augmentent jusqu'à dépasser, un jour, les coûts du distribué.

L'essentiel
Le monolithe est une unité de déploiement unique, simple et cohérente, redoutablement efficace pour démarrer. Ses limites sont des coûts de coordination, de déploiement et de montée en charge qui croissent avec la taille du système et des équipes.

Exemple — Le monolithe SahelMarché à ses débuts

```
SAHELMARCHE - LE MONOLITHE (une seule application)

+--------------------------------------------------+
| Interface web et mobile (pages et ecrans) |
+--------------------------------------------------+
| Modules metier, appels directs en memoire : |
| - Comptes clients - Catalogue produits |
| - Panier et commandes - Paiement (OM / Wave) |
| - Livraison - Notifications SMS |
+--------------------------------------------------+
| Une seule base de donnees (tables partagees) : |
| clients | produits | commandes | paiements ... |
+--------------------------------------------------+

Deploiement : UN seul artefact, produit une fois,
installe tel quel sur le serveur de production.
Exemple : corriger une virgule dans le module
Livraison => reconstruire et redeployer TOUT,
catalogue et paiement compris.
```

À retenir

- Monolithe = une seule unité de déploiement ; cela ne dit rien de la qualité interne du code.

• Ses forces : simplicité de développement, transactions fortes, tests et exploitation économiques.

• Ses limites croissent avec le nombre de développeurs et les besoins de montée en charge ciblée.

Exercice

Une équipe de quatre développeurs lance une application de réservation de salles pour une université : 3 000 utilisateurs, une livraison par semaine, un seul serveur loué. Un collègue propose de « commencer directement en microservices, comme les grands ». Rédige en quelques lignes l'avis que tu donnerais en réunion technique : architecture de départ, arguments, et condition qui ferait changer d'avis plus tard.

✔ Corrigé

Avis : démarrer en monolithe modulaire. Arguments : avec quatre développeurs, les services créeraient plus de travail (réseau, déploiements multiples, supervision) qu'ils n'en retirent ; le trafic est faible et prévisible ; les frontières métier d'un produit neuf sont encore floues, et déplacer du code entre modules d'un monolithe coûte quelques heures, contre des semaines entre services. Le monolithe doit néanmoins être modulaire dès le premier jour : un module par domaine (Comptes, Réservations, Notifications), des appels entre modules par des interfaces explicites, pas d'accès direct aux tables des autres modules. Condition de révision : si l'équipe double, si un module exige une montée en charge très différente des autres, ou si les livraisons se bloquent mutuellement, on réévaluera l'extraction du module le plus autonome. L'architecture suit la douleur réelle, pas l'anticipation.

Leçon 2 — Découper un système en services : domaines et frontières

Découper un système en microservices, c'est d'abord découper le métier, pas la technique. L'approche de référence vient de la conception pilotée par le domaine (Domain-Driven Design, Eric Evans) : on identifie les sous-domaines de l'activité — vendre, encaisser, livrer, fidéliser — et l'on trace autour de chacun un contexte délimité (bounded context), c'est-à-dire une frontière à l'intérieur de laquelle un modèle et un vocabulaire précis ont cours. Le langage ubiquitaire, partagé entre développeurs et gens du métier, est l'outil de ce découpage : si les mêmes personnes emploient les mêmes mots pour parler d'une zone du système, cette zone est probablement un contexte cohérent. C'est ce travail d'écoute du métier, mené avant toute décision technique, qui distingue un découpage durable d'un découpage fait à la mode.

Un même mot peut cacher deux modèles différents, et c'est le signe le plus fiable d'une frontière. Chez SahelMarché, « produit » désigne pour l'équipe Catalogue une fiche vendable : photos, description, prix affiché. Pour l'équipe Stock, le même mot désigne un article physique rangé dans un entrepôt : emplacement, quantité, péremption. Forcer ces deux visions dans un seul modèle « Produit » partagé créerait un objet fourre-tout que personne ne comprend. Deux contextes, deux modèles, et des échanges explicites entre eux : voilà le découpage. D'autres indices aident : un bon

service regroupe ce qui change ensemble, appartient à une seule équipe, et peut être compris sans lire le code des voisins.

Tracer les frontières engage aussi les relations entre contextes : qui dépend de qui, qui impose son modèle à l'autre, où placer une couche de traduction pour protéger un contexte des changements d'un fournisseur externe, par exemple l'API d'un opérateur de paiement mobile. L'erreur symétrique du monolithe géant est le découpage trop fin : des services « nano » qui se téléphonent en permanence pour accomplir la moindre tâche forment un monolithe distribué — le pire des deux mondes, avec la complexité du réseau et le couplage du monolithe. La règle pratique : commencer par quelques contextes larges et bien choisis, quitte à subdiviser plus tard le service qui souffre réellement.

L'essentiel
Un service = un contexte délimité du métier, avec son modèle, son vocabulaire et ses données. La frontière se repère aux changements de sens des mots métier et à ce qui évolue ensemble ; mieux vaut quelques services un peu gros que des nano-services bavards.

Exemple — Carte des contextes de SahelMarché

```
CARTE DES CONTEXTES DE SAHELMARCHE

Contexte Responsabilite Donnees propres
-------------- ----------------------- ----------------
Comptes identite, adresses clients, adresses
Catalogue fiches vendables, prix produits, prix
Stock quantites en entrepot stocks, mouvements
Commandes panier, cycle de commande commandes, lignes
Paiement encaisser OM / Wave transactions
Livraison tournees, suivi de colis expeditions
Notifications SMS et e-mails modeles, envois

Frontieres argumentees :
- "Produit" cote Catalogue = fiche vendable (photo, prix)
- "Produit" cote Stock = article physique (emplacement)
=> deux modeles differents, donc deux contextes.
- Paiement ne lit jamais la base Commandes : il recoit
un ordre de paiement et publie un evenement de resultat.
```

À retenir

- Découper par domaines métier et contextes délimités, jamais par couches techniques (un service « base de données », un service « écrans »).
- Un mot métier qui change de sens d'une équipe à l'autre signale une frontière de contexte.
- Le monolithe distribué — services trop fins et trop couplés — cumule les défauts des deux architectures.

Exercice

Dans une plateforme de formation en ligne, on hésite à fusionner « Cours » et « Facturation » dans un seul service. L'équipe Cours parle de chapitres, de vidéos et de progression des apprenants ; l'équipe Facturation parle d'abonnements, de factures et de relances de paiement. Les deux équipes livrent à des rythmes très différents. Tranche et justifie en utilisant les notions de la leçon.

✔ Corrigé
Il faut deux contextes distincts. D'abord, les vocabulaires ne se recouvrent presque pas : « cours » pour l'un est un contenu pédagogique, pour l'autre une simple ligne de facture désignée par un code — c'est le signal classique de deux modèles différents. Ensuite, la cohésion interne de chaque zone est forte et le couplage entre elles faible : la Facturation a seulement besoin de savoir qu'un abonnement est actif et à quel tarif, information qu'un événement « AbonnementSouscrit » ou un appel ponctuel lui fournit. Enfin, les rythmes de livraison différents confirment l'autonomie des équipes : fusionner obligerait l'équipe Cours à attendre les fenêtres de déploiement de la Facturation. La frontière passe donc entre les deux, avec un contrat explicite : la Facturation publie l'état des abonnements, le service Cours le consomme sans jamais lire la base de facturation.

Leçon 3 — Communication entre services : synchrone et par messages

En mode synchrone, un service appelle un autre service et attend sa réponse, typiquement par une requête HTTP vers une API REST, parfois par gRPC pour plus d'efficacité. C'est le mode le plus simple à comprendre et à déboguer : la demande et la réponse se succèdent dans le même fil d'exécution, et une erreur remonte immédiatement à l'appelant. Chez SahelMarché, quand la page produit a besoin du prix et du niveau de stock pour s'afficher, une lecture synchrone est naturelle. Mais ce mode crée un couplage dans le temps : si le service appelé est lent ou en panne, l'appelant est bloqué ou en échec, et les pannes se propagent en cascade d'un service à l'autre.

En mode asynchrone, l'émetteur dépose un message ou publie un événement, puis continue son travail sans attendre. Un courtier de messages (RabbitMQ, Apache Kafka, ou la file d'un fournisseur cloud) stocke le message et le délivre aux services intéressés, qui le traitent à leur rythme. L'événement énonce un fait passé — « CommandeCreee », « PaiementConfirme » — plutôt qu'un ordre. Ce découplage change tout : le service de notification peut être en maintenance pendant que les commandes continuent d'arriver ; le courtier absorbe les pics d'activité comme un tampon. En échange, le résultat n'est pas immédiat, et suivre le parcours complet d'un traitement demande les outils d'observabilité de la leçon 6.

Le choix se fait donc au cas par cas, selon la question posée. Ai-je besoin de la réponse pour continuer ? Alors synchrone. Est-ce que je signale simplement que quelque chose s'est produit ? Alors événement. Deux pièges guettent le mode asynchrone : la livraison est en général « au moins une fois », donc un consommateur doit être idempotent — traiter deux fois le même message ne doit pas produire deux fois l'effet, par exemple expédier deux colis ; et l'ordre des messages n'est garanti que de façon

limitée, ce que le métier doit accepter. SahelMarché applique la règle suivante : lectures d'affichage en synchrone, tout ce qui déclenche un travail en arrière-plan — SMS, préparation de colis, statistiques — en événements.

L'essentiel
Synchrone : simple et immédiat, mais couplé dans le temps et fragile en cascade. Asynchrone par messages : découplé et absorbant les pics, mais différé, potentiellement livré plusieurs fois — les consommateurs doivent être idempotents.

Exemple — L'événement « CommandeCreee » publié par SahelMarché

```
{
"type": "CommandeCreee",
"version": 1,
"idEvenement": "evt-2026-0417-8f3a",
"idCorrelation": "req-7d21c9",
"dateEmission": "2026-04-17T10:32:05Z",
"emetteur": "service-commandes",
"donnees": {
"idCommande": "CMD-2026-1582",
"idClient": "CLI-0091",
"montantTotal": 48500,
"devise": "XOF",
"lignes": [
{ "idProduit": "PRD-104", "quantite": 2 },
{ "idProduit": "PRD-207", "quantite": 1 }
]
}
}

Qui consomme cet evenement ?
- Stock : reserve les articles
- Notifications : envoie le SMS de confirmation
- Statistiques : met a jour le tableau de bord
Le service Commandes, lui, n'attend personne.
```

À retenir

- Synchrone pour les lectures et les réponses indispensables ; événements pour signaler les faits et déclencher des travaux.
- Livraison « au moins une fois » : tout consommateur asynchrone doit être idempotent.
- Un événement énonce un fait passé (« PaiementConfirme »), il ne donne pas d'ordre aux autres services.

Exercice

Pour chacune de ces quatre interactions de SahelMarché, choisis synchrone ou asynchrone et justifie en une phrase : 1) vérifier le mot de passe d'un client qui se connecte ; 2) envoyer le SMS « votre colis est parti » ; 3) afficher le prix d'un produit sur sa fiche ; 4) recalculer chaque nuit le classement des meilleurs vendeurs.

✔ Corrigé
1) Synchrone : la connexion ne peut pas se terminer sans la réponse — le client attend devant son écran, et un refus doit arriver immédiatement. 2) Asynchrone : la commande est déjà confirmée, le SMS est un travail d'arrière-plan ; s'il part avec deux minutes de retard ou après une reprise du service, personne ne s'en aperçoit. 3) Synchrone : le prix fait partie de la page demandée, il faut la valeur à jour au moment de l'affichage, et le service Catalogue répond en quelques millisecondes. 4) Asynchrone, même planifié : ce calcul lourd et non urgent est typiquement déclenché par un message ou une tâche de fond ; le faire en synchrone bloquerait inutilement un service pendant le traitement. Critère commun : a-t-on besoin de la réponse pour terminer l'action en cours, oui ou non ?

Leçon 4 — Gestion des données dans un système distribué

Le principe fondateur est simple à énoncer : chaque service possède ses données et est le seul à écrire dans sa base. Les autres services n'y accèdent jamais directement ; ils demandent l'information par une API ou écoutent les événements du service propriétaire. Ce principe, « une base de données par service », protège l'autonomie : l'équipe Stock peut changer le schéma de ses tables sans casser le service Commandes. Il a un prix immédiat : il n'existe plus de requête SQL qui joigne les commandes et les produits, ni de transaction unique qui couvre plusieurs services. Toute la leçon découle de ce prix, et il faut le regarder en face plutôt que le découvrir en production.

Dans un monolithe, « réserver le stock ET encaisser le paiement ET confirmer la commande » tient en une transaction : tout réussit ou tout est annulé. En distribué, chaque service ne connaît que sa transaction locale. Le patron saga remplace la transaction globale par une chorégraphie d'étapes locales : chaque étape qui réussit publie un événement qui déclenche la suivante ; si une étape échoue, on exécute les compensations des étapes déjà accomplies — libérer le stock réservé, rembourser le paiement. Une compensation n'est pas un retour arrière magique : un SMS déjà parti ne se « dé-envoie » pas, on en envoie un autre qui rectifie. La saga peut être chorégraphiée (chaque service réagit aux événements) ou orchestrée (un chef d'orchestre pilote les étapes), ce qui se pilote et se débogue plus facilement.

La conséquence la plus déroutante est la cohérence éventuelle : pendant un court instant, le système peut montrer des états intermédiaires — la commande « en attente » alors que le paiement est déjà passé. Il faut l'assumer et la concevoir : écrans qui affichent un statut honnête, aucun calcul critique sur une donnée fraîchement copiée. Le théorème CAP aide à raisonner juste : lorsqu'une coupure réseau (partition) sépare les machines — et elle finit toujours par arriver — un système doit choisir entre rester cohérent (refuser certaines écritures pour ne jamais diverger) et rester disponible (accepter les écritures quitte à réconcilier plus tard). CAP ne dit pas « choisissez deux propriétés sur trois en permanence » : le choix ne se pose qu'au moment de la partition. Le reste du temps, c'est surtout la latence qui arbitre.

L'essentiel
Base par service, donc pas de transaction globale simple : la saga enchaîne des transactions locales et des compensations. La cohérence devient éventuelle, avec des états intermédiaires visibles qu'il faut assumer dans la conception des écrans et des processus.

Exemple — La saga « passer une commande » chez SahelMarché

```
SAGA "PASSER UNE COMMANDE" (orchestration)

Etape Action locale Compensation si echec
----- -------------------------- ------------------------
1 Commandes : creer la (rien a compenser)
commande "en attente"
2 Stock : reserver les Liberer la reservation
articles du stock
3 Paiement : encaisser Rembourser le client
le montant via OM / Wave
4 Commandes : marquer -
la commande "confirmee"

Si l'etape 3 echoue (paiement refuse) :
-> compensation de l'etape 2 : stock libere
-> la commande passe a "annulee"
-> un SMS explique la situation au client

Chaque etape ne touche QUE sa propre base.
Personne ne "defait" le passe : on compense.
```

À retenir

- Une base par service : aucun service ne lit ni n'écrit dans la base d'un autre.
- Saga = suite de transactions locales + compensations en cas d'échec ; une compensation est une nouvelle action métier, pas un effacement.
- CAP : le choix cohérence contre disponibilité ne se pose que pendant une partition réseau.

Exercice

La saga de SahelMarché avance d'une étape : après le paiement confirmé, le service Livraison crée l'expédition et attribue un livreur. Un soir, ce service tombe en panne pendant deux heures alors que les paiements continuent. Décris ce que voient les clients, ce que le système doit faire pendant la panne, et la compensation à prévoir si la livraison s'avère finalement impossible pour une commande payée.

✔ Corrigé
Pendant la panne, les commandes payées restent dans l'état honnête « payée, en attente d'expédition » : l'étape Livraison est déclenchée par l'événement « PaiementConfirme », conservé par le courtier, donc aucun dossier n'est perdu ; les clients voient ce statut expliqué dans leur suivi, pas une erreur. À la reprise, le service consomme les événements en retard et crée les expéditions. Si la livraison est impossible (zone non desservie, adresse introuvable), la saga exécute ses compensations dans l'ordre inverse : annuler l'expédition prévue, libérer le stock réservé, puis rembourser le paiement — chaque remboursement étant lui-même une transaction du service Paiement, déclenchée par événement. Le client reçoit un SMS d'explication et le remboursement sur son compte mobile money : on ne peut pas « dé-payer », on compense proprement.

Leçon 5 — Résilience : tolérance aux pannes et files d'attente

Dans un système distribué, la question n'est pas de savoir si un composant tombera en panne, mais quand, et ce que les autres feront à ce moment-là. Un appel réseau peut échouer, mais aussi — plus traître — réussir côté serveur et perdre sa réponse en route : l'appelant ne sait pas si l'action a eu lieu. Les premiers réflexes sont donc défensifs : fixer un délai d'attente (timeout) à chaque appel, car attendre indéfiniment bloque des ressources en chaîne ; réessayer les échecs temporaires avec un nombre de tentatives limité et un délai croissant agrémenté d'un peu d'aléatoire (back-off exponentiel avec jitter), pour ne pas achever un service convalescent sous un déluge de nouvelles tentatives synchronisées.

Le disjoncteur (circuit breaker) protège l'ensemble : il compte les échecs d'un service appelé ; au-delà d'un seuil, il « ouvre » le circuit et fait échouer les appels immédiatement, sans même tenter le réseau, pendant une durée de repos. Le service malade respire, l'appelant récupère vite une réponse — quitte à servir une donnée de repli : le niveau de stock approximatif plutôt qu'une page d'erreur. Après le repos, le disjoncteur laisse passer quelques appels de test (état semi-ouvert) et ne se referme que si la santé est revenue. Autour gravitent d'autres patrons du même esprit : limiter les fils d'exécution dédiés à chaque dépendance (cloisonnement, comme les compartiments étanches d'un navire) et dégrader gracieusement les fonctions non essentielles en période de pointe.

Les files d'attente sont le deuxième grand outil de résilience, côté asynchrone. En plaçant une file entre l'émetteur et le consommateur, on découple leurs destins : si le service de notifications tombe, les messages s'accumulent sagement et seront traités à son retour, au lieu de faire échouer les commandes. La file absorbe aussi les pics : pendant les soldes, SahelMarché accepte toutes les commandes et laisse les consommateurs rattraper leur retard progressivement. Reste le cas du message « poison », qui échoue à chaque tentative : après un nombre fixé d'essais, il est mis de côté dans une file des échecs (dead letter queue) où un humain l'examinera, au lieu de bloquer éternellement la file principale.

L'essentiel
En distribué, la panne est un cas normal : délais d'attente partout, réessais limités et espacés, disjoncteur qui coupe court pour protéger l'ensemble, files d'attente qui découplent les destins et absorbent les pics, file des échecs pour les messages impossibles à traiter.

Exemple — Une file d'attente protège l'envoi des SMS

```
FILE D'ATTENTE : LES SMS DE CONFIRMATION

Service File (courtier) Service
Commandes +---------------+ Notifications
--------- | sms.a_envoyer | -------------
publie le ----> | messages en | ----> consomme a son
message | attente | rythme
+-------+-------+ si panne :
| le message RESTE
v en file, rien
+---------------+ n'est perdu
| File des |
| echecs (DLQ) | apres 5 tentatives :
+---------------+ analyse humaine

Sans file : panne du service SMS = commandes en echec,
alors que le paiement, lui, a reussi.
Avec file : commande confirmee, SMS des la reprise.
```

À retenir

- Aucun appel réseau sans délai d'attente ; tout réessai est limité, espacé et croissant (back-off avec jitter).
- Le disjoncteur fait échouer vite pour protéger le système entier et laisse le service malade récupérer.
- Une file découple les destins : le consommateur peut tomber et revenir sans perdre de messages.

Exercice

Le service Paiement de SahelMarché devient très lent pendant 15 minutes (réponses en 20 secondes au lieu de 200 millisecondes), sans être complètement en panne. Explique ce qui arrive au service Commandes s'il appelle le paiement en synchrone sans délai d'attente ni disjoncteur, puis décris le comportement avec un délai d'attente de 2 secondes et un disjoncteur réglé à 50 % d'échecs sur 20 appels.

✔ Corrigé
Sans protection, chaque commande ouverte garde un fil d'exécution bloqué 20 secondes en attendant le paiement : les fils s'accumulent, le service Commandes sature à son tour et cesse de répondre même pour les pages qui n'ont rien à voir avec le paiement — c'est la panne en cascade, où un service lent contamine des services sains. Avec un délai d'attente de 2 secondes, chaque appel libère son fil rapidement : la commande est marquée « paiement à confirmer » au lieu d'échouer silencieusement, et le traitement se termine de façon asynchrone à la réponse tardive ou par réconciliation. Avec le disjoncteur, dès que la moitié des 20 derniers appels échouent ou expirent, le circuit s'ouvre : les appels suivants sont refusés instantanément pendant la période de repos, le service Commandes reste disponible pour tout le reste, et Paiement récupère sans être martelé. Après le repos, quelques appels de test vérifient la guérison avant la réouverture complète.

Leçon 6 — Observabilité d'un système distribué

Déboguer un monolithe est confortable : une seule pile d'appels, un seul journal, et l'erreur se trouve presque toujours à quelques lignes de l'endroit où elle éclate. Dans un système de services, une seule action d'utilisateur traverse cinq machines, trois files et deux bases de données ; quand le client déclare « ma commande a échoué », la cause peut se cacher n'importe où. L'observabilité est la capacité à comprendre l'état interne d'un système depuis ses sorties. Elle repose sur trois piliers complémentaires : les journaux (logs), qui racontent les événements un par un ; les métriques, qui mesurent en continu (débit, taux d'erreurs, temps de réponse) ; et les traces, qui suivent le voyage complet d'une requête à travers les services.

La pratique qui change tout tient en une phrase : chaque requête reçoit, dès son entrée dans le système, un identifiant unique — l'identifiant de corrélation — que chaque service recopie dans tous ses journaux et transmet dans tous ses appels et messages. Dès lors, reconstituer l'histoire d'une commande devient une simple recherche par identifiant dans l'agrégateur de journaux, au lieu d'une fouille archéologique serveur par serveur. Pour que cela marche, les journaux doivent être structurés (des champs nommés plutôt que du texte libre), émis vers un collecteur central, et débarrassés des données sensibles : un journal qui contient des mots de passe ou des numéros complets de carte est une faille de sécurité, pas un outil de débogage.

Les métriques et les alertes forment le tableau de bord vivant du système : pour chaque service, on surveille au minimum le débit des requêtes, le taux d'erreurs et les durées de réponse — les signaux « RED » — ainsi que la saturation des ressources (processeur, mémoire, files qui s'allongent). Une alerte bien conçue signale un symptôme qui affecte les utilisateurs (« 5 % des paiements échouent depuis dix minutes »), pas chaque cause technique possible, et elle arrive avec l'identifiant de corrélation d'un exemple d'échec pour commencer l'enquête. Enfin, la trace distribuée assemble les segments de chaque service en une frise unique qui montre où le temps s'est perdu. Observabilité n'est pas un luxe de grande entreprise : c'est la condition pour exploiter plus de quelques services sans voler à l'aveugle.

L'essentiel
Trois piliers : journaux structurés, métriques, traces. Le geste fondateur est l'identifiant de corrélation, créé à l'entrée et propagé partout, qui permet de reconstituer le voyage complet de chaque requête et de passer du symptôme à la cause en quelques minutes.

Exemple — Cinq lignes de journal, une seule requête

```
JOURNAUX CORRELES - requete req-7d21c9 (un achat)

10:32:05.112 commandes INFO req-7d21c9
commande CMD-2026-1582 creee (48 500 XOF)
10:32:05.240 stock INFO req-7d21c9
reservation OK : PRD-104 x2, PRD-207 x1
10:32:05.388 paiement WARN req-7d21c9
tentative 1 refusee par l'operateur
10:32:05.913 paiement INFO req-7d21c9
tentative 2 acceptee, ref OM-88412077
10:32:05.940 commandes INFO req-7d21c9
commande passee a "confirmee"

Le meme identifiant sur les 5 lignes, dans 3 services
differents : on relit l'histoire complete de l'achat.
Sans lui, ces lignes seraient perdues parmi des
milliers d'autres, et la panne serait introuvable.
```

À retenir

- Journaux = le récit, métriques = les mesures en continu, traces = le voyage d'une requête entre services.
- L'identifiant de corrélation est créé à l'entrée du système et propagé dans chaque appel et chaque message.
- Des alertes sur les symptômes visibles par les utilisateurs, jamais de données sensibles dans les journaux.

Exercice

Chez SahelMarché, le taux d'abandon du paiement augmente brutalement un mardi à 14 h, sans déploiement ce jour-là. Tu disposes des trois piliers d'observabilité. Décris ta démarche d'enquête dans l'ordre : quel signal consultes-tu d'abord, que cherches-tu ensuite, et comment la trace d'une commande échouée te mène-t-elle à la cause ?

✔ Corrigé
D'abord les métriques : le tableau de bord du service Paiement confirme le symptôme (taux d'échecs passé de 1 % à 30 % à 14 h) et le délimite — les autres services sont normaux, et le délai de réponse de l'opérateur mobile money a triplé sur le même graphique. Ensuite les journaux : on filtre par service Paiement et par la fenêtre horaire, en regroupant par motif d'erreur ; la mention dominante « délai dépassé chez l'opérateur » apparaît des centaines de fois, assortie des identifiants de corrélation des commandes touchées. Enfin la trace : on suit une de ces commandes de bout en bout — Commandes en 40 ms, Stock en 60 ms, puis 25 secondes d'attente chez Paiement avant l'échec. La cause est donc externe et identifiée (lenteur de l'opérateur), ce qui oriente la réponse : allonger temporairement le délai d'attente du paiement, basculer les commandes concernées en « paiement à confirmer » plutôt qu'en échec, et prévenir le support avec des chiffres précis à transmettre à l'opérateur.

Leçon 7 — Étude de cas : migration progressive d'un monolithe

SahelMarché a grossi : vingt-cinq développeurs, des soldes qui saturent le catalogue, et des livraisons devenues un cérémonial hebdomadaire risqué. L'équipe décide de migrer — mais pas en réécrivant tout le système d'un coup. La réécriture complète est le scénario le plus dangereux de notre métier : elle gèle les nouveautés pendant des mois, sous-estime systématiquement les fonctions cachées de l'ancien système, et livre un remplacement qui découvre ses propres bogues le jour du basculement général. La sagesse du domaine, formalisée par Martin Fowler sous le nom de patron de l'étrangleur (strangler fig, comme le figuier qui enveloppe progressivement son arbre hôte), propose l'inverse : faire pousser le nouveau système autour de l'ancien, fonction par fonction, jusqu'à ce que l'ancien n'ait plus de raison d'exister.

La mécanique commence par une façade : on place une passerelle (API gateway) devant le monolithe, de sorte que tout le trafic des clients passe par elle. Au départ, elle transmet tout au monolithe, sans rien changer. Ensuite, à chaque extraction, on détourne une route de la passerelle vers le nouveau service ; les clients ne voient aucune différence. L'ordre d'extraction suit deux critères : commencer par le service le moins couplé — chez SahelMarché, les Notifications, qui ne font que réagir à des événements — pour apprendre le distribué à faible risque ; puis attaquer le service qui souffre le plus, le Catalogue, dont la lecture massive étouffe le serveur pendant les campagnes. Le cœur transactionnel, Commandes et Stock, sort en dernier, quand l'équipe maîtrise déjà sagas, files et supervision.

Le point délicat de chaque extraction est la donnée : le nouveau service a besoin des informations restées dans la base du monolithe. On procède par étapes prudentes — copie initiale des données, puis période de double lecture comparée (on lit dans l'ancien, on vérifie contre le nouveau) ou de double écriture temporaire, avant de basculer définitivement les lectures puis les écritures, avec un retour arrière possible à chaque étape. La migration de SahelMarché s'étale ainsi sur plusieurs mois, système en production en permanence, et elle peut même s'arrêter en route : si après trois extractions les douleurs ont disparu, conserver un cœur monolithique modulaire

entouré de services est un aboutissement respectable, pas un échec.

L'essentiel
Migrer progressivement, jamais par réécriture totale : une passerelle devant le monolithe, des extractions une à une en commençant par le moins couplé, des données migrées avec doubles lectures vérifiées, et le système en production à chaque instant.

Exemple — Le plan de migration de SahelMarché en six étapes

```
PLAN DE MIGRATION PROGRESSIVE (patron de l'etrangleur)

Etape 1 Passerelle API placee devant le monolithe :
tout le trafic la traverse, rien ne change.
Etape 2 Extraire Notifications (le moins couple) :
le monolithe publie des evenements, le nouveau
service envoie les SMS. Risque faible, equipe
qui apprend le distribue en douceur.
Etape 3 Extraire Catalogue (celui qui souffre) :
copie des donnees, doubles lectures comparees,
puis bascule des lectures vers le service.
Etape 4 Extraire Paiement : mise en place de la saga
avec les Commandes encore dans le monolithe.
Etape 5 Extraire Commandes et Stock, en dernier :
le coeur transactionnel, quand l'equipe
maitrise sagas, files et supervision.
Etape 6 Retirer le code mort du monolithe, ou le
conserver s'il sert encore utilement.

A CHAQUE etape : production en service, retour
arriere possible, aucune "grande bascule".
```

À retenir

- La réécriture complète est le scénario le plus risqué : on étrangle le monolithe fonction par fonction.
- La passerelle détourne le trafic route par route ; chaque extraction est réversible.
- On extrait d'abord le moins couplé, puis ce qui souffre le plus, et le cœur transactionnel en dernier.

Exercice

Une autre plateforme, « LibrairiePlus », vend des livres en ligne avec un monolithe devenu lent. Son équipe te présente son plan : arrêter le site un week-end entier, tout réécrire en douze microservices pendant ce week-end, et basculer le lundi matin. Rédige ta critique du plan (trois risques précis) et l'alternative que tu proposes pour le premier mois.

✔ Corrigé
Trois risques précis : 1) le délai — douze services en un week-end suppose que les frontières sont déjà connues et justes, alors qu'on ne les découvre presque jamais sans expérimenter ; tout retard déborde sur l'ouverture du lundi, site fermé ou site bancal. 2) La donnée — migrer toutes les tables d'un coup, sans double lecture ni vérification, transforme la moindre erreur de conversion en perte définitive, découverte par les clients. 3) Le tout-ou-rien — un seul service défaillant le lundi (un contrat mal compris, une saga incomplète) bloque l'ensemble des ventes, sans retour arrière possible puisque l'ancien système a été éteint. Alternative pour le premier mois : placer une passerelle devant le monolithe (semaine 1), instrumenter journaux corrélés et métriques pour savoir ce qui souffre vraiment (semaine 2), extraire le service le moins couplé, par exemple les notifications d'expédition (semaines 3-4), site ouvert en permanence et retour arrière à chaque étape.

Travaux dirigés (TD)

TD 1 — Faut-il découper ? Trois situations à trancher

Pour chacune de ces trois situations, décide : monolithe simple, monolithe modulaire, ou microservices — et justifie par les coûts et les bénéfices dans ce contexte précis. Situation A : deux étudiants lancent un service de petites annonces pour leur campus, budget d'hébergement minimal, fonctionnalités encore incertaines. Situation B : une banque mobile emploie 120 développeurs en 12 équipes, déploie chaque jour, et chaque équipe se plaint d'attendre les autres pour livrer. Situation C : une PME de 8 développeurs gère un logiciel de paie stable, qui change deux fois par an pour suivre la réglementation, avec des calculs qui exigent une cohérence parfaite entre salaires, cotisations et déclarations.

✔ Corrigé du TD 1
Situation A : monolithe simple. Deux personnes, des frontières métier inconnues, un budget serré : les microservices y coûteraient en infrastructure et en temps exactement ce qui manque le plus. Une seule application, une seule base, un déploiement ; on structure proprement le code pour garder l'avenir ouvert. Situation B : microservices justifiés. La douleur dominante est organisationnelle — 12 équipes bloquées les unes par les autres — et c'est précisément le problème que les services autonomes résolvent ; la banque a la taille d'équipe, la cadence de livraison et les moyens d'assumer l'exploitation distribuée, l'observabilité et les sagas. Situation C : monolithe modulaire, et c'est le cas le plus intéressant. Huit développeurs qui livrent deux fois par an ne souffrent ni de coordination ni de montée en charge ; en revanche, la paie exige des transactions fortes entre calculs liés — exactement ce qu'une base unique offre gratuitement et ce qu'une saga rendrait pénible et risqué. La bonne réponse y est donc : un monolithe découpé en modules clairs (Salaires, Cotisations, Déclarations), pas des services. Leçon générale : on ne choisit pas une architecture, on choisit quels coûts on préfère payer.

TD 2 — Autopsie d'un incident distribué

Chez SahelMarché, un jeudi soir, des clients paient mais ne reçoivent ni confirmation ni SMS, et le service Commandes affiche « paiement en cours » pendant des heures pour 300 commandes. L'équipe dispose des journaux corrélés, des métriques et de la file des échecs. Reconstitue un scénario d'incident cohérent avec ces symptômes, explique comment chaque outil d'observabilité aide à le confirmer, et décris la résolution immédiate puis la correction définitive.

✔ Corrigé du TD 2
Scénario cohérent : le service Paiement a reçu un déploiement à 18 h qui contient un défaut — il encaisse bien l'argent chez l'opérateur, mais il échoue ensuite au moment de publier l'événement « PaiementConfirme » (par exemple à cause d'un champ renommé dans le message). Les commandes restent donc « en cours » côté Commandes, qui attend l'événement ; les SMS, déclenchés par la confirmation, ne partent jamais ; l'argent, lui, est bien débité — d'où la gravité. Confirmation par les outils : les métriques montrent le taux d'erreurs de publication de Paiement qui grimpe à 18 h précises, alors que les encaissements réussissent ; les journaux corrélés d'une commande cliente montrent « paiement accepté, référence OM… » suivi d'une erreur de sérialisation du message ; la file des échecs contient les messages « PaiementConfirme » rejetés avec la même erreur. Résolution immédiate : retour arrière du service Paiement vers la version précédente, puis retraitement des messages de la file des échecs pour confirmer les 300 commandes et déclencher les SMS — grâce à l'idempotence des consommateurs, le retraitement ne crée ni double commande ni double débit. Correction définitive : versionner les schémas d'événements avec des tests de contrat dans le pipeline (le cours INF-L3-12), ajouter une alerte qui compare encaissements réussis et confirmations publiées — tout écart durable déclenche une alerte — et consigner la procédure de retraitement dans le carnet d'exploitation.

Travaux pratiques (TP)

TP 1 — Concevoir la cible d'une plateforme de location de voitures

« BamakoCars », plateforme fictive de location de voitures entre particuliers, fonctionne aujourd'hui en monolithe et son équipe de dix développeurs veut préparer une cible en services. Sans écrire de code, produis le dossier d'architecture : 1) la carte des contextes délimités (au moins cinq contextes) avec, pour chacun, sa responsabilité en une phrase, ses données propres et son équipe ; 2) un tableau des communications : pour chaque interaction entre contextes, synchrone ou par événement, avec la justification ; 3) la saga complète de « réserver une voiture » (étapes, événements, compensations) ; 4) le plan de migration en quatre étapes ordonnées, avec le critère qui fait passer d'une étape à la suivante.

✔ Attendus du TP 1
Dossier attendu (une réponse modèle parmi d'autres défendables). 1) Contextes : Comptes (identité des locataires et propriétaires, pièces vérifiées) ; Flotte (les voitures proposées, leurs caractéristiques et photos) ; Disponibilités (calendriers et blocages par voiture) ; Réservations (cycle de vie d'une location, de la demande à la restitution) ; Paiement (caution et règlement, remboursements) ; Notifications (SMS et e-mails aux deux parties). La séparation Disponibilités/Réservations se justifie : « voiture » y a deux sens, calendrier d'un côté, contrat de l'autre. 2) Communications : lecture d'une fiche voiture et de ses disponibilités = synchrone (la page en a besoin tout de suite) ; création d'une réservation = commande synchrone courte qui démarre la saga ; validations, refus, rappels = événements (« ReservationConfirmee », « CautionRemboursee ») consommés par Notifications, Flotte et Statistiques. 3) Saga : Réservations crée la demande « en attente » ; Disponibilités bloque les dates (compensation : libérer les dates) ; Paiement prélève la caution (compensation : rembourser) ; Réservations confirme et publie l'événement ; si le paiement échoue, on libère les dates et la demande passe à « refusée » avec notification. 4) Migration : étape 1, passerelle et observabilité (critère : tout le trafic passe par la passerelle, journaux corrélés en place) ; étape 2, extraction des Notifications (critère : un mois sans perte de message) ; étape 3, extraction de Flotte + Disponibilités en lecture (critère : doubles lectures concordantes à 99,9 %) ; étape 4, extraction de Réservations et Paiement avec la saga (critère : retraitements d'incident testés en environnement de recette).

Exercices corrigés supplémentaires

Exercice 1

Un soir de match, le réseau qui relie les deux centres de données de SahelMarché est coupé pendant 20 minutes. Le service Commandes doit continuer à vendre sur le centre A, le service Stock vit sur le centre B. En utilisant le théorème CAP énoncé correctement, explique le choix qui se pose pendant la coupure, et ce qui se passe à la reconnexion selon le choix retenu (cohérence d'abord, ou disponibilité d'abord).

✔ Corrigé
Pendant la partition, les deux centres ne peuvent plus se synchroniser : c'est exactement le moment où CAP s'applique. Choix « cohérence » : le service Commandes refuse les ventes des produits dont le stock ne peut être vérifié — certaines commandes échouent, mais jamais on ne vend un article épuisé ; à la reconnexion, il n'y a rien à réconcilier. Choix « disponibilité » : chaque centre continue de vendre sur la base de sa dernière copie du stock ; on accepte donc un risque de survente, et à la reconnexion il faut réconcilier : comparer les journaux de ventes, détecter les articles vendus deux fois et traiter les exceptions (remboursement, produit équivalent, excuses au client). CAP ne tranche pas à notre place : il dit que pendant la coupure, il faudra payer l'un des deux prix. Hors partition, les deux centres resynchronisés offrent les deux propriétés, moyennant la latence des échanges.

Exercice 2

Un service A appelle un service B en synchrone. B met normalement 100 ms à répondre, mais un jour sur cinq, il met 30 secondes. On te propose trois réglages pour A : (a) aucun délai d'attente, aucun réessai ; (b) délai d'attente de 30 secondes et 5 réessais immédiats ; (c) délai d'attente de 1 seconde, 3 réessais avec délais de 1 s, 2 s, 4 s et un peu d'aléatoire, plus un disjoncteur. Classe ces réglages du plus dangereux au plus sain et explique le classement.

✔ Corrigé
Le plus dangereux est (a) : sans délai d'attente, chaque appel lent bloque un fil d'exécution jusqu'à 30 secondes ; les fils s'épuisent et A tombe en cascade, y compris pour ses fonctions sans rapport avec B. Vient ensuite (b) : le délai de 30 secondes ne protège de rien (c'est exactement la durée du problème), et 5 réessais immédiats multiplient par 6 la charge sur un B déjà en difficulté — les réessais synchronisés de tous les clients achèvent le malade ; c'est l'effet « troupeau tonnerre ». Le plus sain est (c) : le délai court libère vite les ressources d'A ; les réessais espacés de façon exponentielle et légèrement aléatoire laissent à B le temps de respirer et désynchronisent les clients ; le disjoncteur, si les échecs persistent, coupe les appels pour protéger A et laisse B récupérer en paix. Règle générale : un réessai sans espacement ni limite est une attaque contre le service qu'on prétend aider.

Exercice 3

Le service Statistiques de SahelMarché reçoit l'événement « CommandeCreee » deux fois pour la même commande (livraison « au moins une fois » du courtier) et compte donc la vente en double dans le chiffre d'affaires du jour. Propose deux façons de rendre ce consommateur idempotent, et explique pourquoi la solution « le courtier ne doit jamais livrer deux fois » n'est pas recevable.

✔ Corrigé
Première façon : déduplication par identifiant — le consommateur enregistre les idEvenement déjà traités (dans une table de sa propre base, écrite dans la même transaction que la mise à jour des statistiques) ; à la redélivrance, il reconnaît l'identifiant et acquitte le message sans rien recalculer. Deuxième façon : rendre le traitement naturellement idempotent en recalculant depuis la source — par exemple, l'événement transporte le montant de la commande et Statistiques remplace la contribution de cette commande (clé = idCommande) au lieu de l'additionner : traiter deux fois le même fait revient à écrire deux fois la même valeur, résultat identique. La troisième voie — exiger du courtier une livraison « exactement une fois » — n'est pas recevable : entre la panne du consommateur après traitement mais avant acquittement, aucun protocole ne peut distinguer « pas traité » de « traité mais non confirmé » ; les courtiers sérieux garantissent donc « au moins une fois » et laissent l'idempotence au consommateur, qui seul connaît le sens métier de « déjà fait ».

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle phrase décrit correctement un monolithe ?

A) Une application mal écrite, par définition

B) Une application déployée comme une seule unité, dont les modules s'appellent en mémoire

C) Une application sans base de données

2. Quel est l'indice le plus fiable d'une frontière de contexte délimité ?

A) Un même mot métier change de sens d'une équipe à l'autre

B) Le nombre de tables de la base dépasse cinquante

C) Le langage de programmation diffère entre équipes

3. Une équipe de trois développeurs lance un produit nouveau aux frontières encore floues. Le choix le plus raisonnable est…

A) Douze microservices dès le départ

B) Aucune architecture : coder au fil de l'eau

C) Un monolithe modulaire, dont les modules pourront être extraits plus tard

4. Un événement « PaiementConfirme » publié par le service Paiement est…

A) Un ordre donné aux autres services

B) L'énoncé d'un fait passé, que chaque service intéressé consomme à son rythme

C) Une requête qui attend une réponse immédiate

5. Dans une architecture à base de données par service, comment le service Livraison apprend-il qu'une commande est payée ?

A) En consommant l'événement « PaiementConfirme » ou en interrogeant l'API du service propriétaire

B) En lisant directement la table des paiements

C) En partageant la même base que Commandes

6. À quoi sert une compensation dans une saga ?

A) À annuler physiquement l'écriture dans toutes les bases

B) À redémarrer les services en échec

C) À exécuter une nouvelle action métier qui neutralise l'effet d'une étape déjà accomplie (rembourser, libérer)

7. Que dit exactement le théorème CAP ?

A) Un système doit choisir deux propriétés sur trois, pour toujours

B) Pendant une partition réseau, il faut arbitrer entre cohérence et disponibilité

C) Les bases NoSQL sont toujours incohérentes

8. Un disjoncteur « ouvert » a pour effet de…

A) Faire échouer immédiatement les appels vers le service malade, sans tenter le réseau

B) Supprimer les messages de la file

C) Redémarrer automatiquement le service en panne

9. À quoi sert l'identifiant de corrélation ?

A) À chiffrer les échanges entre services

B) À authentifier l'utilisateur sur chaque service

C) À relier entre elles toutes les lignes de journal produites par une même requête, à travers les services

10. Dans le patron de l'étrangleur, quelle est la première étape typique ?

A) Éteindre le monolithe un week-end pour tout réécrire

B) Placer une passerelle devant le monolithe afin de pouvoir détourner le trafic route par route

C) Extraire d'abord le cœur transactionnel, le plus critique

Corrigés du QCM

1. Réponse B — Le monolithe est une forme de déploiement ; il peut être proprement modulaire à l'intérieur.

2. Réponse A — Le changement de sens d'un terme (« produit » fiche vendable contre article en entrepôt) révèle deux modèles distincts.

3. Réponse C — Les frontières bougent trop au début ; déplacer du code entre modules coûte peu, entre services beaucoup.

4. Réponse B — L'événement constate un fait ; il ne commande personne et n'attend pas de réponse.

5. Réponse A — Aucun accès direct aux données d'un autre service : événement ou API, jamais la base du voisin.

6. Réponse C — On ne défait pas le passé : on compense par une action inverse au niveau métier.

7. Réponse B — Le choix ne se pose qu'au moment d'une coupure réseau ; le reste du temps, cohérence et disponibilité coexistent.

8. Réponse A — Il protège l'appelant et laisse le service défaillant récupérer, avant des appels de test en semi-ouvert.

9. Réponse C — Créé à l'entrée et propagé partout, il permet de reconstituer le voyage complet d'une requête.

10. Réponse B — La façade rend chaque extraction progressive et réversible ; le cœur transactionnel, lui, sort en dernier.

Résumé — les notions importantes

• Le monolithe est une unité de déploiement simple, cohérente et économique ; ses limites sont des coûts de coordination et de montée en charge qui croissent avec la taille des équipes et du trafic.

• Un microservice correspond à un contexte délimité du métier, avec son modèle, son vocabulaire et ses données ; les frontières se lisent dans le langage des équipes, pas dans la technique.

• Synchrone pour les réponses indispensables, asynchrone par événements pour découpler et absorber les pics — avec des consommateurs idempotents, car la

livraison est « au moins une fois ».

• Base de données par service : la transaction globale disparaît, remplacée par la saga (transactions locales + compensations) et une cohérence éventuelle assumée.

• CAP : pendant une partition réseau seulement, il faut arbitrer entre cohérence et disponibilité ; le reste du temps, c'est la latence qui coûte.

• La résilience se construit : délais d'attente, réessais espacés et limités, disjoncteur, files d'attente et file des échecs pour les messages impossibles.

• L'observabilité (journaux structurés, métriques, traces, identifiant de corrélation) et la migration par l'étrangleur rendent le distribué exploitable au quotidien — sans elles, rester monolithique est souvent le choix le plus professionnel.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Architectures monolithiques : forces et limites
Monolithe = une seule unité de déploiement ; cela ne dit rien de la qualité interne du code. ; Ses forces : simplicité de développement, transactions fortes, tests et exploitation économiques. ; Ses limites croissent avec le nombre de développeurs et les besoins de montée en charge ciblée.

Fiche 2 — Découper un système en services : domaines et frontières
Découper par domaines métier et contextes délimités, jamais par couches techniques (un service « base de données », un service « écrans »). ; Un mot métier qui change de sens d'une équipe à l'autre signale une frontière de contexte. ; Le monolithe distribué — services trop fins et trop couplés — cumule les défauts des deux architectures.

Fiche 3 — Communication entre services : synchrone et par messages
Synchrone pour les lectures et les réponses indispensables ; événements pour signaler les faits et déclencher des travaux. ; Livraison « au moins une fois » : tout consommateur asynchrone doit être idempotent. ; Un événement énonce un fait passé (« PaiementConfirme »), il ne donne pas d'ordre aux autres services.

Fiche 4 — Gestion des données dans un système distribué
Une base par service : aucun service ne lit ni n'écrit dans la base d'un autre. ; Saga = suite de transactions locales + compensations en cas d'échec ; une compensation est une nouvelle action métier, pas un effacement. ; CAP : le choix cohérence contre disponibilité ne se pose que pendant une partition réseau.

Fiche 5 — Résilience : tolérance aux pannes et files d'attente
Aucun appel réseau sans délai d'attente ; tout réessai est limité, espacé et croissant (back-off avec jitter). ; Le disjoncteur fait échouer vite pour protéger le système entier et laisse le service malade récupérer. ; Une file découple les destins : le consommateur peut tomber et revenir sans perdre de messages.

Fiche 6 — Observabilité d'un système distribué
Journaux = le récit, métriques = les mesures en continu, traces = le voyage d'une requête entre services. ; L'identifiant de corrélation est créé à l'entrée du système et propagé dans chaque appel et chaque message. ; Des alertes sur les symptômes visibles par les utilisateurs, jamais de données sensibles dans les journaux.

Fiche 7 — Étude de cas : migration progressive d'un monolithe
La réécriture complète est le scénario le plus risqué : on étrangle le monolithe fonction par fonction. ; La passerelle détourne le trafic route par route ; chaque extraction est réversible. ; On extrait d'abord le moins couplé, puis ce qui souffre le plus, et le cœur transactionnel en dernier.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Monolithe vs Micro-services : Ce que tu DOIS Comprendre

Chaîne : Coding With Armand · Langue : Français · Niveau : Intermédiaire
Une comparaison directe des deux architectures, avec leurs avantages et leurs coûts réels, qui prolonge la leçon 1. Idéale pour fixer les critères de choix avant d'attaquer le découpage en domaines.

https://www.youtube.com/watch?v=heL-X6OPw5w

Leçon 2 — Formation Gratuite Complète Microservices et application Cloud-Native + labs

Chaîne : eazytraining · Langue : Français · Niveau : Intermédiaire à avancé
Une formation longue et pratique qui part du monolithe, découpe en microservices et va jusqu'à l'orchestration et la supervision. À regarder par chapitres, en commençant par ceux sur le découplage et la conteneurisation.

https://www.youtube.com/watch?v=hpSpwH8K-xg

Leçon 3 — Apache Kafka : comprendre l'essentiel en 4 minutes

Chaîne : Cookie connecté · Langue : Français · Niveau : Débutant à intermédiaire
Quatre minutes pour comprendre à quoi sert Kafka, le courtier d'événements le plus cité dans les architectures distribuées. Le complément parfait de la leçon sur la communication par messages, avant d'aller plus loin.

https://www.youtube.com/watch?v=kCLjBcKrbfM

Leçon 7 — Back-end : passer du monolithe aux microservices | Exemple concret

Chaîne : RichLabo · Langue : Français · Niveau : Intermédiaire
Le récit concret d'un passage du monolithe aux microservices sur un vrai projet, avec les compromis rencontrés en chemin. À comparer point par point avec le plan de migration de SahelMarché de la leçon 7.

https://www.youtube.com/watch?v=4IO4i_2wUtA

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Sam Newman — « Building Microservices » (O'Reilly) : l'ouvrage de référence sur la conception et l'évolution des systèmes de microservices.

• Eric Evans — « Domain-Driven Design : Tackling Complexity in the Heart of Software » (Addison-Wesley) : le livre fondateur des contextes délimités et du langage ubiquitaire.

• Chris Richardson — « Microservices Patterns » (Manning) et le site microservices.io : sagas, CQRS et décomposition, avec des exemples détaillés.

• Martin Fowler — articles sur les architectures distribuées et le patron de l'étrangleur (martinfowler.com, en accès libre).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !